Reviewer Pack — Operator Norm Lower Bound for Disjointness Communication Mat...

Entry 4da9869ae44e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 00:30:56 UTC

Operator Norm Lower Bound for Disjointness Communication Matrices

Entry ID: `4da9869ae44e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 00:30:56 UTC

1. Conjecture

Field A (mathematical branch): Operator Spaces Theory **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For any n×n matrix M representing the DISJOINTNESS function, the completely bounded norm ||M||_{cb} ≥ Ω(n). This holds for all matrices where M[i,j] = 1 iff the i-th and j-th subsets are disjoint, with random subset generation using uniform distribution over 2^{⌈log₂n⌉}-element sets.

Rationale (proposer's reasoning):

> Operator spaces theory provides tools to analyze tensor norms that capture the structure of communication matrices. The cb-norm, being invariant under tensor products, could expose inherent dimensionality constraints in DISJOINTNESS that classical discrepancy methods miss.

Taxonomy category: `COMM_DISJ` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `87c199b2ed0d6df4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_disjointness_matrix(n):
    subsets = [set(random.sample(range(1, 2**math.ceil(math.log2(n))), n)) for _ in range(n)]
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if len(subsets[i].intersection(subsets[j])) == 0:
                M[i][j] = 1
                M[j][i] = 1
    return M

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def identity_matrix(n):
    I = [[0] * n for _ in range(n)]
    for i in range(n):
        I[i][i] = 1
    return I

def numerical_range(M, tol=1e-6):
    n = len(M)
    I = identity_matrix(n)
    B = [M[i] - (i+1)*I for i in range(n)]
    eigenvalues = []
    for k in range(100):  # Max iterations
        v = [random.random() for _ in range(n)]
        v /= sum(v) ** 0.5
        w = matrix_multiplication(M, v)
```

```
        lambda_k = sum(w[i] * v[i] for i in range(n))
        eigenvalues.append(lambda_k)
    return max(eigenvalues), min(eigenvalues)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    M = generate_disjointness_matrix(n)
    cb_norm_max, _ = numerical_range(M)
    metric_value = cb_norm_max
    instances_tested = 1
    conjecture_holds = metric_value >= 0.1 * n
    counterexample = "" if conjecture_holds else "cb_norm < 0.1*n"
    return {
        "metric_name": "cb_norm",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric = sum(r['metric_value'] for r in results) / len(results)
    std_metric = (sum((r['metric_value'] - mean_metric)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric:.6f} std={std_metric:.6f} support_fraction={su
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric:.6f} std={std_metric:.6f} support_fraction={su
    else:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"cb_norm < 0.1*n\" first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc72256d.py", line 78, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc72256d.py", line 60, in run_trial
    cb_norm_max, _ = numerical_range(M)
```

```
                    ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc72256d.py", line 46, in numerical_ran
    B = [M[i] - (i+1)*I for i in range(n)]
         ~~~~~^~~~~~~~~
TypeError: unsupported operand type(s) for -: 'list' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to TypeError in matrix operations, preventing result collection | next: Fix the numerical_range function's matrix subtraction logic to handle list operations properly

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109421
2	propose	ollama_remote	qwen3:8b	0	0	101248
3	propose	ollama_remote	qwen3:8b	0	0	40463
4	preregistration	ollama_remote	qwen3:8b	0	0	24086
5	novelty	ollama_remote	qwen3:8b	0	0	20773
6	novelty	ollama_remote	qwen3:8b	0	0	18453
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12791
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10314
9	judge	ollama_remote	qwen3:8b	0	0	19015

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 356564 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4da9869ae44e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4da9869ae44e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4da9869ae44e.tar.gz` (if generated)